You said:

I agree its settled. Lets talk Main!

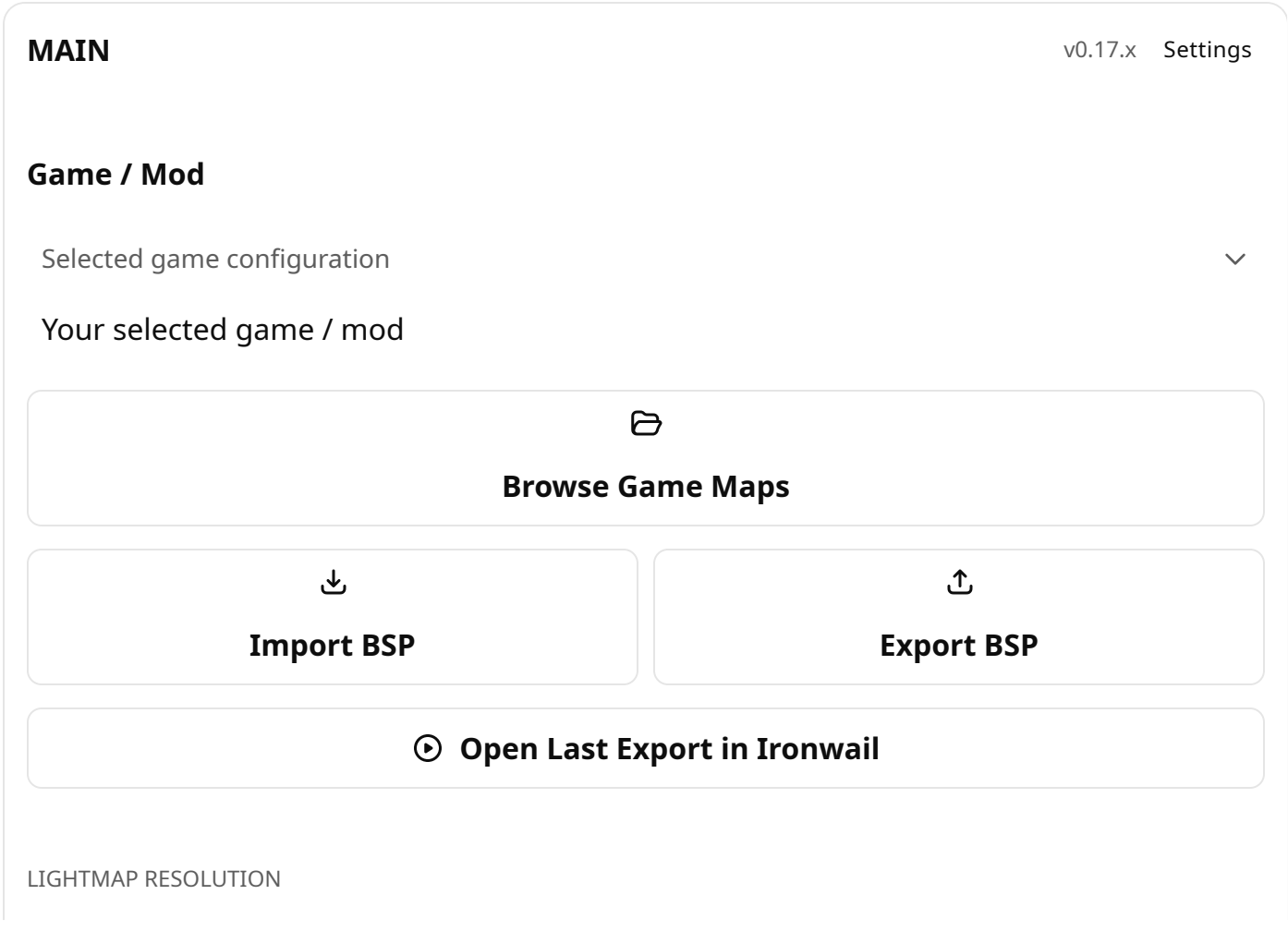
ChatGPT said:

Good. Let's treat **MAIN** as the central project-control section of QSS. It should answer four questions without making the user hunt through dialogs:

1. Which game/mod am I working with?
2. Which map am I working on?
3. Can I import, export, or launch it?
4. What lightmap resolution is the project using?

I'd keep your proposed layout, with a few behavioural details to settle before touching the code.

2. MAIN — proposed structure



16 units / luxel Change

Conceptual layout, not a screenshot of the implemented Blender panel.

Behaviour I'd propose

Version + Settings

- Version number stays visible.
- Settings opens the QSS configuration file in Blender's Text Editor.
- Ideally, it opens in a separate area or window without disrupting the 3D View.
- Invalid or missing configuration should produce a clear warning, not silently fall back to misleading values.
- Settings-dependent controls remain disabled until their required configuration is valid.

One important technical distinction: editing the configuration file and applying its changes are separate operations. We should provide a clear reload/apply mechanism rather than assuming every text edit immediately updates QSS.

Game / Mod

- Keep the existing game/mod selector.
- It determines the game directory, maps directory, engine executable and other game-specific paths.
- If a required path is missing, show which one needs attention.

Browse Game Maps

- Opens the existing map browser.
- Selecting a map should make it the current project map.
- Import should still be possible independently.

Import BSP

- Opens a deliberately simple file selector.
- No extra QSS settings in the file selector.
- On successful import:
  - Set the imported BSP as the active project.
  - Move the viewport to `info_player_start`.
  - Enable project-dependent controls.

- Set lighting display to Original.

Export BSP

I agree with your automatic preset idea. The important part is that the automatic choice should be a *default*, not a restriction.

- Classic: export standard BSP.
- Enhanced: export enhanced BSP.
- Both: export both variants, with `_c` and `_e` suffixes.
- Choose the default based on whether the current lightmap resolution requires enhanced output.
- Let the user override the choice in the export dialog.

Open Last Export in Ironwail

- Opens the last successfully exported file.
- If Both was selected, prefer `_e` by default.
- Allow that preference to be changed in config.
- Disable the button until a valid export exists.

Lightmap Resolution

- Always display the current project resolution.
- Change opens a popup with the established resolution choices and warnings.
- Changing resolution should update the export default.
- We should make it clear that changing the resolution does not magically rebake existing lightmaps.

Four details I'd like to settle

There are a few decisions that affect the underlying implementation, so I'd rather agree on them now.

1. Settings file editing

- ○ Open the actual config file in Blender's Text Editor; user saves it manually.
- ○ Use a QSS settings popup with proper controls.

- ○ Both: quick settings popup plus advanced config file editing.

2. Browse Game Maps

- ○ Browse maps and select one; importing remains a separate action.
- ○ Selecting a map immediately imports it.

3. Importing while another map is already loaded

- ○ Replace the current map after confirmation.
- ○ Import as a separate map collection in the same scene.

4. Export format default

- ○ Automatically choose Classic or Enhanced from lightmap resolution; Both remains an explicit option.
- ○ Remember the last export choice, with automatic choice only as the initial default.

Confirm these decisions

My preference is the combination shown by the first option in each group, except that I think **Both** should always remain an easy explicit choice. Once these decisions are settled, we can finalise MAIN and implement it without disturbing the navigation section.

You said:

For the MAIN section, my decisions are:
Settings: both
Map browser: import
Import behaviour: new
Export default: auto
Let's refine the remaining MAIN details.

Stopped thinking

You said:

About points 3 and 4:
Importing an additional map into its own map collection is what i would prefer but we should have a popup asking wheter we want to replace and also default settings (add / replace / ask) in the config. Same for 4. Automatic should be the default since it chooses the best for the map and gives us the option to change it in export dialogue. Anyways default settings in the config wont hurt. :D Good work. Lets keep on.

ChatGPT said:

Exactly. That gives us flexibility without sacrificing safe defaults. And I agree: **adding another map should never silently destroy an existing one.** :D

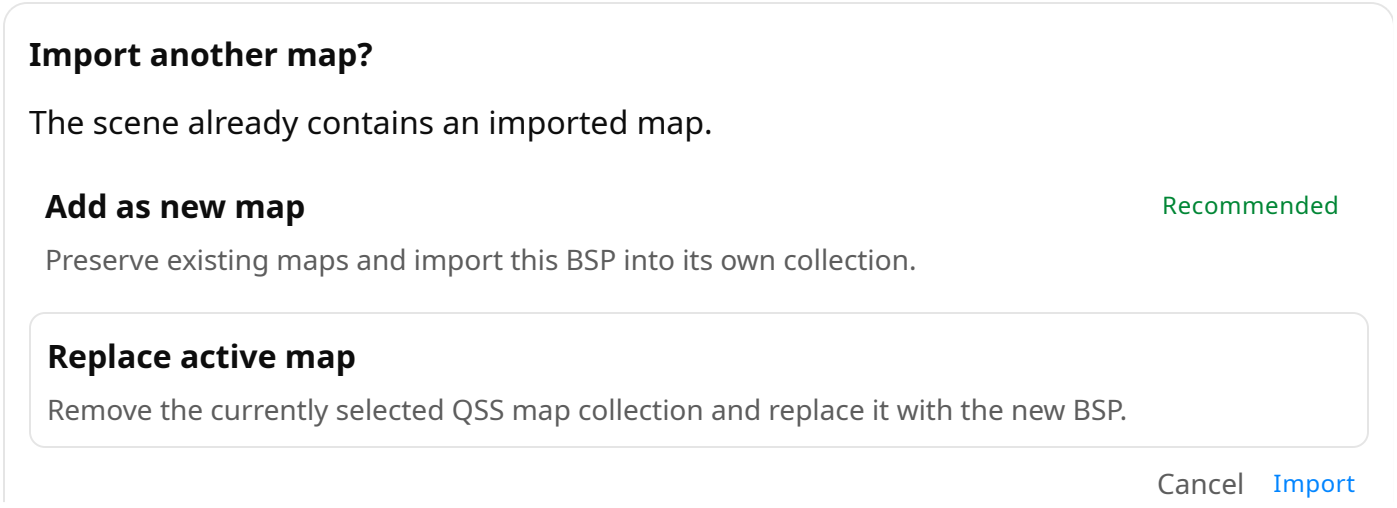
Let's lock those decisions into the MAIN specification.

Import behaviour

The three configuration options will be:

Config value	Behaviour
`ADD`	Import into a new, separate map collection.
`REPLACE`	Replace the currently selected QSS map collection.
`ASK`	Show a popup asking what to do.

My recommendation for the initial configuration is `ASK`, with **Add** preselected in the popup.



Illustrative import confirmation, not an implemented Blender dialog.

A few important rules:

- Add creates a uniquely named collection and makes the newly imported map active.
- Replace only affects the currently active QSS map collection, not unrelated scene objects.
- Cancel leaves the scene untouched.
- If there is no existing QSS map, import directly without asking.
- The configuration can switch between Add, Replace and Ask at any time.

Export behaviour

Here I would use the same principle.

The configuration gets:

```
export_mode = AUTO
```

Available values:

- `AUTO`
- `CLASSIC`
- `ENHANCED`
- `BOTH`

The export dialogue starts with the configured mode. With AUTO, QSS evaluates the current map and lightmap settings and determines the appropriate format.

Export Quake BSP

Output format Automatic

Automatic — Enhanced
Selected based on this map's current lightmap requirements.

Available formats

- Automatic

- Classic
- Enhanced
- Both (Classic + Enhanced)

The user can override the automatic choice for this export.

For Both, we produce two files:

```
my_map_c.bsp
my_map_e.bsp
```

And the Ironwail button:

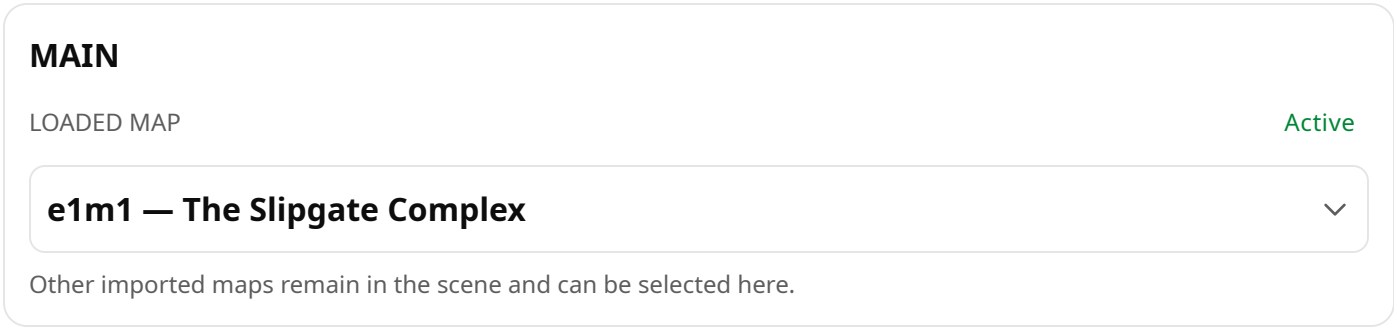
- Opens the last successfully exported BSP.
- Prefers `_e` when the last export was Both.
- Has a configuration setting to change that preference.
- Uses the actual exported path, rather than trying to reconstruct a filename.

The automatic format decision should be based on QSS's actual BSP/lightmap capability rules, not an arbitrary new threshold.

One additional thing we need in MAIN: active map selection

Since we are now allowing multiple maps in one scene, we need a way to tell QSS which map the other operations should target.

I propose a **Loaded Map** dropdown, visible once at least one BSP is imported.



This is important because export, lightmap resolution, lighting and baking must all know which map they are operating on. Selecting a map should also make its collection the active QSS project, without requiring the user to reorganize anything in the Outliner.

One question to settle this part: Should importing a new map automatically make it the active map?

My recommendation is yes. It is the natural workflow: import, arrive at its player start, and immediately work on that map. Existing maps remain safely in the scene.

You said:

I agree! Yes it should become the active one. Im happy with the main section. Lets plan the next.

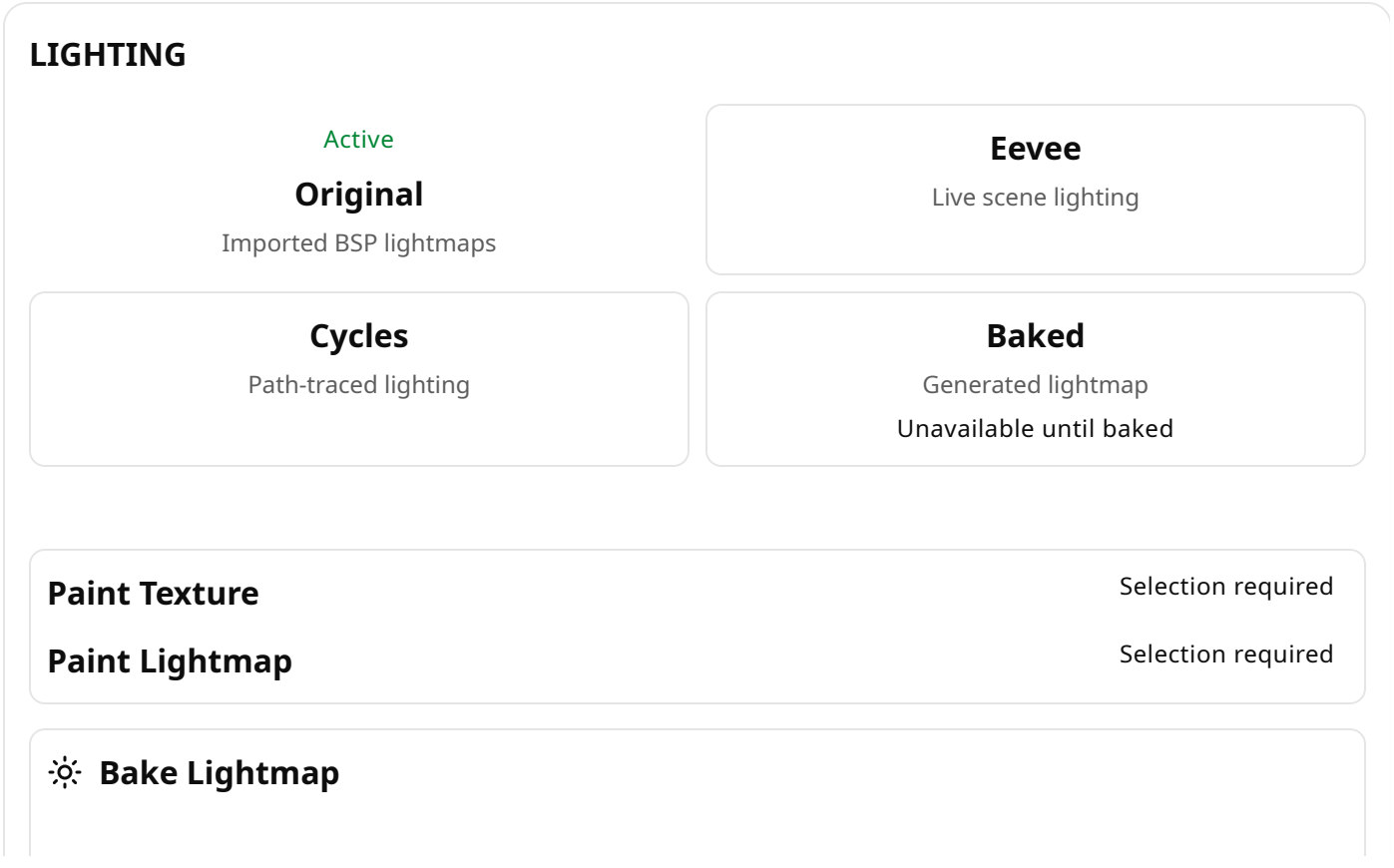
Worked for 44s

ChatGPT said:

Then LIGHTING is our next major section. I think the four modes and three action buttons are a good foundation. The challenge is making them behave consistently without changing the actual lighting data or accidentally overwriting the original BSP lightmaps.

I'd keep your proposed layout and refine its behaviour as follows.

3. LIGHTING — proposed design



Available when a supported live-lighting mode is active.

A. The four lighting modes

Original

This should be the default after importing a BSP.

- Displays the original imported BSP lightmaps.
- Disables the influence of Blender scene lights and world lighting.
- Represents the original Quake lighting as closely as possible.
- Does not modify the imported lightmap images.

Eevee

- Displays live lighting from Blender's scene.
- Disables the original BSP lightmap contribution.
- Uses the EEVEE rendering engine.
- Allows fast lighting adjustments and interactive feedback.

Cycles

- Switches to Cycles and rendered viewport shading.
- Enables scene lights and world lighting.
- Disables the original BSP lightmap contribution.
- Provides a more accurate lighting preview.

Baked

- Displays the generated QSS lightmap instead of the original BSP lightmap.
- Uses the same lightmap-oriented display behaviour as Original.
- Remains unavailable until the active map has a valid baked lightmap.

One important technical adjustment: Blender's Solid viewport mode uses the Workbench engine and does not evaluate material shader nodes. Therefore, it cannot reliably display QSS's image-based lightmaps or actual Eevee scene lighting. We should use a shader-aware viewport mode for these presets, rather than force everything into Solid. Blender 5.2 ... +2

The visual result and intended behaviour matter more than the name of Blender's viewport shading mode.

B. Paint Texture and Paint Lightmap

I would preserve the behaviour you described.

Situation	Behaviour
No selected faces	Both buttons disabled
Faces selected in Edit Mode	Both buttons enabled
Click Paint Texture	Enter Texture Paint
Click Paint Lightmap	Enter Lightmap Paint
Click the currently active paint button	Leave paint mode
Click the other paint button	Switch paint target/mode

Most importantly, switching between texture and lightmap painting must not lose the selected-face mask or unexpectedly change the active map.

We should also ensure that Paint Lightmap targets the intended lightmap image, never whichever image texture node happens to be active in the material.

C. Bake Lightmap

Here I found one important Blender limitation that we need to account for.

Blender's documented Render Bake workflow for baking shader lighting into image textures is the **Cycles** workflow. EEVEE supports light-probe baking, but that is not the same thing as baking a UV lightmap for our BSP surfaces. Blender 5.2 ... +1

I don't want to make the Eevee button appear functional while secretly producing a Cycles bake. That would make the two modes misleading.

My proposal:

- Bake creates a separate lightmap layer for the active map.
- The imported original lightmaps remain untouched.
- Re-baking updates the generated layer, with confirmation before replacing existing results.
- Successful baking enables Baked mode.
- Each imported map has its own baked-lightmap status and data.

For the first implementation, we should use the reliable Cycles baking pipeline. We can investigate a genuine EEVEE-to-lightmap solution separately before promising that capability.

Three decisions to settle

1. Eevee baking

Should we keep investigating a proper Eevee bake implementation, even though the initial reliable bake will use Cycles?

- ○ Yes. I want genuine Eevee baking eventually.
- ○ No problem. Cycles-only baking is fine.

2. Re-baking an existing lightmap

- ○ Ask before replacing the existing baked layer.
- ○ Overwrite it immediately.
- ○ Keep every bake as a separate version.

3. Lighting mode selection

When switching between maps, should QSS remember each map's last selected lighting mode?

- ○ Yes. Each map remembers its own mode.
- ○ No. One lighting mode applies globally to all maps.

Confirm lighting decisions

My preference is the first option in all three questions. In particular, per-map lighting state makes sense now that MAIN supports multiple maps in one Blender scene.

Once this is settled, we can finish the specification for LIGHTING before making any changes to the add-on.

You said:

Lighting decisions:
Eevee baking: cycles
Re-baking: confirm
Lighting mode state: per-map
Let's finalise the Lighting section.

Stopped thinking

You said:

Youre right! I confused the solid mode with the material preview mode wich is basically eevee! :D Good catch.

About Eevee baking: It would be nice and it cant hurt to do it eventually but i would also be able to live without it. Maybe implement it naively as you would with cycles as a test but lets not focus to much ernergy into it for now. For the rest as in main: I prefer the same options ays you but making them changable in the config would be good.

Worked for 1m 44s

ChatGPT said:

Haha, yes! That distinction changes the design quite a bit. You're right: **Material Preview is essentially Blender's EEVEE-based viewport renderer**, using an HDRI environment by default. It also has separate controls for using the scene's lights and world. Blender 5.2 ... +1

That actually gives us a much cleaner solution. Let's consider the LIGHTING section finalised.

Final LIGHTING specification

1. Lighting presets

Preset	Viewport behaviour	Lightmap source
Original	Material Preview; Scene Lights and Scene World off	Original BSP lightmaps

Eevee	Material Preview; Scene Lights and Scene World on	No BSP lightmaps
Cycles	Rendered viewport; Cycles engine; scene lighting enabled	No BSP lightmaps
Baked	Material Preview; Scene Lights and Scene World off	QSS-generated lightmaps

Baked is disabled until the active map has a valid generated lightmap.

Each map remembers its own selected lighting preset. Switching maps restores that map's previous state.

2. Painting controls

```
[Paint Texture]
[Paint Lightmap]
```

- Both require selected faces in Edit Mode.
- Clicking an inactive button enters the corresponding paint mode.
- Clicking the active button exits paint mode.
- Clicking the other button switches directly between painting targets.
- The selected faces and active map must be preserved.
- Paint Lightmap always targets the active map's intended lightmap image.

3. Bake Lightmap

```
[Bake Lightmap]
```

The bake creates a separate generated lightmap layer for the active map.

Cycles: The supported, dependable baking path.

Eevee: We will try a straightforward experimental implementation. It is a nice capability to investigate, but it must not hold up the rest of the lighting system. Blender's documented UV image-texture baking workflow is based on Cycles, whereas EEVEE's light-probe baking is a different workflow. Blender 5.2 ... +1

Other rules:

- Original BSP lightmaps are never overwritten.

- Re-baking an existing generated layer asks for confirmation.
- Successful baking makes Baked mode available.
- Failed or unsupported baking reports the problem clearly and preserves existing results.
- Generated lightmaps belong to their respective map collections.

4. Configuration options

These are the proposed defaults, all configurable:

Setting	Default
Lighting mode storage	`PER_MAP`
Re-bake existing lightmap	`ASK`
Eevee baking	`EXPERIMENTAL`
Standard bake engine	`CYCLES`

The important distinction is that the experimental Eevee option is not a promise of equivalent functionality. It simply permits us to investigate it without making the rest of QSS dependent on it.

LIGHTING specification settled

Next: STATS

This should be the smallest section and shouldn't become another control panel. I suggest keeping it informational and context-sensitive.

Some useful candidates:

- Active map name and BSP version.
- Number of faces, textures and materials.
- Number of entities and lights.
- Lightmap resolution and total lightmap memory.
- Original versus generated lightmap status.
- Current map collection and export status.

I'd also like the statistics to tell us something useful about the map, rather than simply repeat values that are already obvious elsewhere.

My guiding principle for STATS: information worth knowing at a glance, without buttons or configuration clutter.

Let's shape that section next.

You said:

Just some small things before stats:

We need a way to enable and disable painted textures.
Also by default importing a map should hide everything exept its collection.

ChatGPT said:

Good additions. Both fit naturally into the existing design, and they become especially important now that we support multiple maps in one scene.

I'd add them without introducing another sidebar section.

1. Painted Textures toggle

Add a control directly below the painting buttons in LIGHTING:

```
[Paint Texture]
[Paint Lightmap]
[Painted Textures: ON]
```

Its behaviour:

- **ON:** Display the painted texture versions.
- **OFF:** Display the original, unpainted textures.
- Switching does not discard or modify either version.
- Applies to all painted textures belonging to the active map.
- Each map remembers its own setting.

Important implementation detail

We need to preserve the original imported textures separately from their painted versions. Otherwise, painting directly into the original image would make this toggle impossible without destructive workarounds.

I'd make **ON** the default. Before anything has been painted, the toggle has no visible effect.

2. Isolate imported map

On successful import:

- Show the newly imported map collection.
- Hide everything else in the 3D viewport, including other map collections and unrelated scene objects.
- Do not delete, move or modify the hidden objects.
- Keep all other maps available in the Outliner.
- Make the imported map active and move the viewport to its player start.

I'd add a configuration option:

```
isolate_map_on_import = true
```

I'd also like to preserve the previous visibility state so that we can eventually provide a reliable way to leave isolation and restore the scene as it was.

One small behaviour to decide

When you select another map in the MAIN **Loaded Map** dropdown, should QSS automatically isolate that map too?

My recommendation is **yes**. That way, the active map always gets the focused viewport, and switching maps feels consistent with importing one.

Everything else in the LIGHTING specification remains unchanged.

You said:

Yes i guess only having one map visible at a time is the way to go. That way they wont light each other. Also they will probably overlap anyways.
Having a way to togle painted textures makes it also interesting to

have the same for painted lighmaps.. I guess the lightmap paint makes mostly sense as an lightmap replacement so at least in original and bake mode it should be togglable. hmmm..

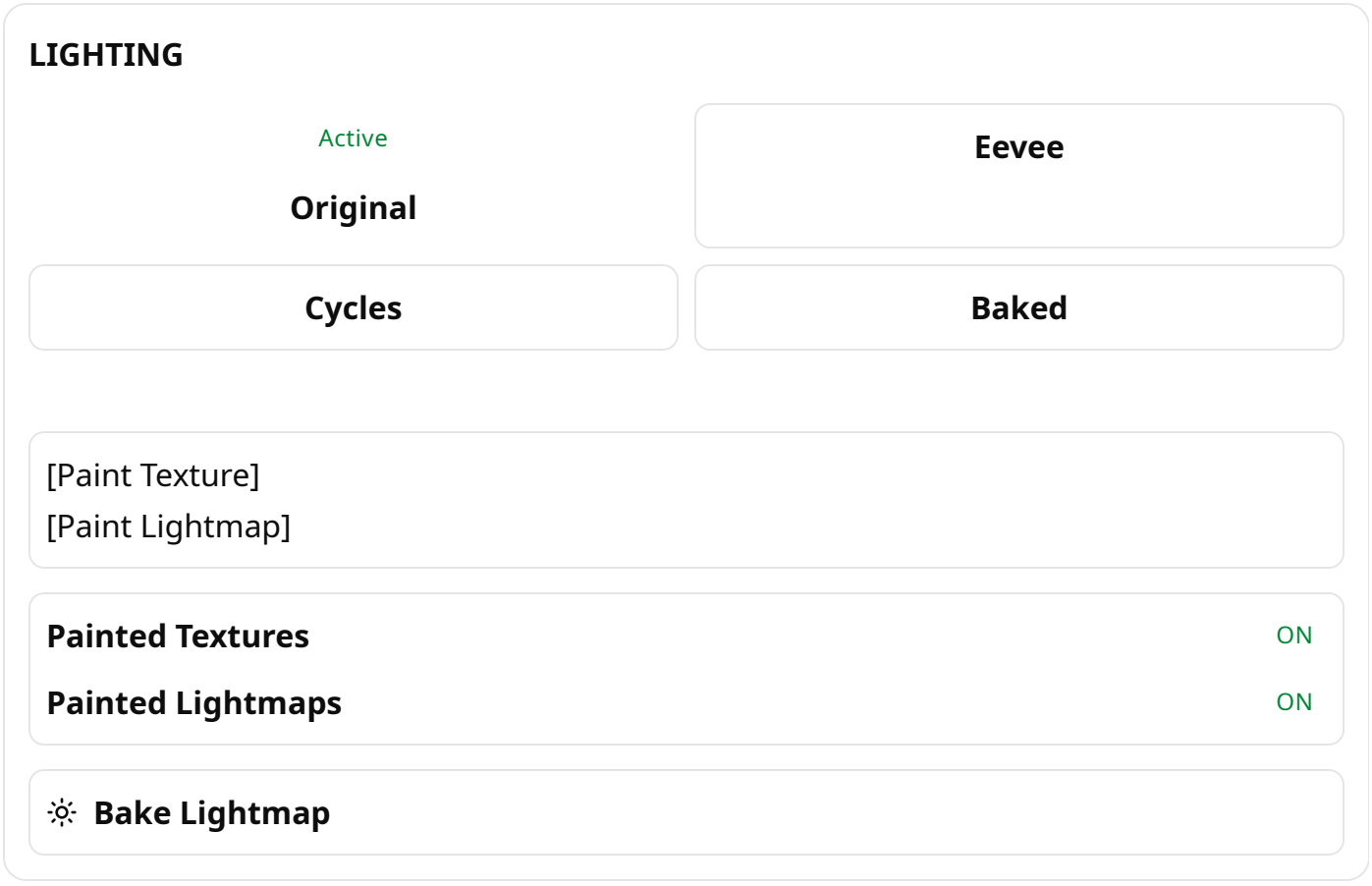
ChatGPT said:

I think you've identified an important distinction: **painted textures and painted lightmaps are two different kinds of overrides.** They should have independent visibility controls.

And yes, painted lightmaps should be useful in both Original and Baked modes. In fact, that makes lightmap painting considerably more interesting.

Updated LIGHTING layout

I'd add two switches beneath the painting buttons:



How I would handle the two switches

Painted Textures

This is independent of the lighting mode.

- ON displays painted texture versions.
- OFF displays original imported textures.
- Works in Original, Eevee, Cycles and Baked.
- Each map remembers its setting.
- Original texture data is preserved.

Painted Lightmaps

This depends on the selected lighting mode.

Mode	Painted Lightmaps switch
Original	Available
Eevee	Disabled
Cycles	Disabled
Baked	Available

The important part is how we store the painting.

I propose two independent painted variants per map:

- **Original lightmap paint:** An editable override of the imported BSP lightmaps.
- **Baked lightmap paint:** An editable override of the generated QSS lightmaps.

That way, painting the original lightmap doesn't accidentally modify the baked result, and vice versa.

In Original mode, the switch selects between the imported lightmap and its painted version. In Baked mode, it selects between the generated bake and its painted version.

`Paint Lightmap` paints whichever of these two sources is currently active.

Re-baking will ask for confirmation. If an existing painted bake depends on the old bake, the confirmation should explicitly warn that the painted version will need to be reset or recreated.

Map visibility — also settled

I agree completely about only displaying one map at a time.

Let's make this a core rule:

- Only the active QSS map collection is visible in the viewport.
- Switching the Loaded Map dropdown switches the visible collection.
- Importing a new map makes it active and isolates it automatically.
- Other collections remain intact in the Outliner.
- No accidental lighting interaction between overlapping maps.

This also gives every map a much more independent working environment.

LIGHTING design complete

I think we've now got a coherent lighting workflow: original assets remain safe, painting is reversible, generated lightmaps remain separate, and every map maintains its own state.

I'd consider LIGHTING settled. Next up is STATS, where we can keep things considerably simpler.

You said:

Yes lets do stats!

ChatGPT said:

Let's make STATS the quiet, informative part of the sidebar. No buttons, no settings and no actions hidden in here. Just useful information about the **currently active map**.

Since we now support multiple maps, every statistic should belong to the selected map, never accidentally combine data from the entire Blender scene.

4. STATS — proposed design

I'd divide the information into four small groups:

STATS

MAP

Map name	**e1m1**
BSP format	**BSP29**

Source file	e1m1.bsp
GEOMETRY & ENTITIES	
BSP faces	**1,284**
Models	**42**
Entities	**86**
Lights	**11**
TEXTURES & LIGHTMAPS	
Materials	**37**
Missing textures	0
Original lightmaps	**12**
Generated lightmap	Not baked
EXPORT	
Last export	Not exported
Last format	—

Illustrative values and layout, not actual map data.

A few design decisions

1. Source statistics versus current scene statistics

This distinction matters. For example, the original BSP face count should remain an accurate record of the imported file. If geometry gets edited in Blender, a separate current-scene count could be useful.

I'd label these clearly rather than silently changing what a statistic means.

2. Useful warnings

I would include small informational warnings here where appropriate:

- Missing textures.

- Invalid or unsupported entities.
- Lightmap data missing or incomplete.
- Export limitations.

These should be concise and non-interactive. Detailed diagnostics can remain elsewhere in the add-on's internal logic, without bringing back another UI section.

3. Export information

Showing the last export is useful because of the new Open Last Export button in MAIN. It lets us see at a glance whether the active map has actually been exported.

4. No invented statistics

If a value cannot be measured reliably, don't display it. In particular, I would rather have six accurate statistics than fifteen questionable ones.

One question before we settle it

Should STATS show only the **imported BSP data**, or should it also reflect changes made to the editable Blender scene?

My recommendation is both, with clear labels where they differ. That would make STATS useful not just as a map information panel, but also as a quick way to understand what has changed since importing.

You said:

Sounds good. Stats is your panel! :D Put stuff there. But maybe make it collapsed on load.

ChatGPT said:

Haha, accepted! :D I'll take responsibility for making STATS useful without turning it into a wall of numbers.

STATS will be collapsed by default. Users can expand it whenever they need more detail. It won't compete visually with NAVIGATION, MAIN or LIGHTING.

Final STATS design

I'll organise it into five groups:

Group	Information
MAP	Map name, source file, BSP version, map dimensions
GEOMETRY	Original BSP face count, current geometry, models, vertices and polygons
ENTITIES	Total entities, entity types, player starts, lights and other notable entities
TEXTURES & LIGHTING	Materials, missing textures, original lightmaps, painted overrides, generated bakes and memory usage where measurable
EXPORT & DIAGNOSTICS	Last export, format, timestamp, warnings, missing or unsupported data

A few principles I'll follow:

- **Original and current values stay distinct.** Editing geometry must not rewrite the original BSP statistics.
- **Map-specific information only.** Nothing gets accidentally aggregated across multiple imported maps.
- **Live where practical.** Scene-dependent values update when the map changes or relevant geometry is edited.
- **Honest measurements.** Unknown or unreliable values are omitted rather than guessed.
- **Warnings are informative.** No buttons or extra workflows hidden in STATS.
- **No unnecessary clutter.** The most useful figures come first when expanded.

Our complete sidebar structure

Quake Surface Studio

NAVIGATION

[WASD Mode]

[Move to Player Start]

MAIN

Version / Settings

Game / Mod

Browse Game Maps

Import BSP / Export BSP

Open Last Export in Ironwail

Lightmap Resolution / Change

Loaded Map

LIGHTING

Original / Eevee / Cycles / Baked

Paint Texture / Paint Lightmap

Painted Textures ON/OFF

Painted Lightmaps ON/OFF

Bake Lightmap

STATS Collapsed by default

Expandable map information, diagnostics and export statistics

Sidebar design agreed

I consider the planning phase complete. The next step is to turn this specification into an implementation plan, then rebuild the sidebar section by section while preserving the working BSP, navigation, painting and lighting functionality. No workspace or toolbar interfaces creeping back in!

You said:

looks good! Good job! The outline is complete! Now impress me by following our plan and implementing it as a working addon and make the next version a pleasure again :D

Worked for 46m 1s

ChatGPT said:

Well, we have a new version. And this time, the sidebar has a proper architecture behind it. :D

QSS v0.18.0 CI passed

The new map-centric sidebar is implemented on our existing feature branch.

Blender-ready installation

Version	**0.18.0**
Build	**GitHub Actions #372**
Package	**Blender Install ZIP**

Get QSS v0.18.0

SHA-256: `490eac5acef0be83e5cad3c52890dce800f1ca54eacbb170eac0e4bf72a013ac`

What's implemented

- **NAVIGATION:** Native noclip WASD navigation and Move to Player Start.
- **MAIN:** Game/Mod configuration, map browser, import prompts, separate map collections, active-map switching, export formats, Ironwail launch and lightmap resolution.
- **LIGHTING:** Four preview modes, automatic face detachment for texture painting, paint-mode switching, independent painted texture/lightmap toggles and Cycles baking.
- **STATS:** Detailed active-map information, diagnostics and export information, collapsed by default.

The important architectural changes are also in place:

- Maps are isolated from one another in both viewport and render visibility.
- Each map retains its own lighting and painting state.
- Importing a new map makes it active.
- Export follows the selected Original or Baked lightmap layer.
- Configuration controls both the quick Settings popup and advanced JSON editing.

Eevee preview works independently; its Bake button currently uses the Cycles backend. A genuine Eevee lightmap bake remains a future experiment.

Validation

All CI steps passed

- Python module compilation
- Standalone regression tests
- Add-on version verification
- Blender package assembly
- Install artifact upload

One important distinction: CI does not run Blender's interactive GUI. I haven't claimed to have verified the actual Blender 5.2 interface or the full multi-map painting workflow.

Your turn: the first GUI test

I'd particularly like you to test these in order:

1. Install v0.18.0 and open the Quake sidebar.
2. Configure the Quake root and Ironwail executable in Settings.
3. Import one BSP, then import another using Add.
4. Switch between the two maps and check their visibility and lighting isolation.
5. Test Paint Texture, Paint Lightmap and both painted-layer toggles.
6. Try export with Automatic and Both.

The PR is updated, still **draft and unmerged**. `main` remains untouched.

View PR #1 — QSS v0.18.0 ↗

I'm especially interested in your first impression of the sidebar in Blender. We've spent quite a while getting the structure right; now let's see whether it feels right in actual use. 😁